● Backend Developer · Available for Work

신승화

Backend Developer / API · DB · Payment Integration

결제 시스템의 위변조를 서버 단에서 차단하고, 외부 API 데이터를 안정적으로 파이프라인화하는 백엔드 개발자입니다.

문제의 근본 원인을 추적하고, 구조적으로 해결하는 것을 중시합니다.

📱 010.7726.7867

✉️ tlstmdghk2@naver.com

🐙 GitHub

01 Profile

교육이수내역

2025.04 – 2025.10
KH 정보교육원 수료
빅데이터 기반 엘라스틱서치 자바 개발자 과정

2005.03 – 2011.02
부산 보건대학교
응급구조과 졸업

2026.03
정보처리기사
필기 합격
2026년 최종 취득 예정

프로젝트 이력

2025.09.10 – 2025.10.20
GEULBUT · 온라인 서점 고도화 메인
JPA + Gradle + Spring Security
PortOne 결제 · 알라딘 API · 마이페이지 SPA

2025.06.30 – 2025.07.22
BLUEMING PAGE · 온라인 서점 보조
XML + Maven + eGovFrame
회원관리 · 도서 검색 · 장바구니 주문

02 Tech Stack

BACKEND

- Java 17
- Spring Boot
- Spring Security
- JPA
- MyBatis
- Python

FRONTEND

- JSP
- HTML / CSS
- Bootstrap
- Swiper.js
- Ajax

DATABASE

- Oracle DB
- Elasticsearch

API & TOOLS

- PortOne
- Aladin OpenAPI
- Git / GitHub
- ERD
- eGovFrame

03 GEULBUT Project

MAIN PROJECT

2025.09.10 – 2025.10.20

GEULBUT

GitHub Repository

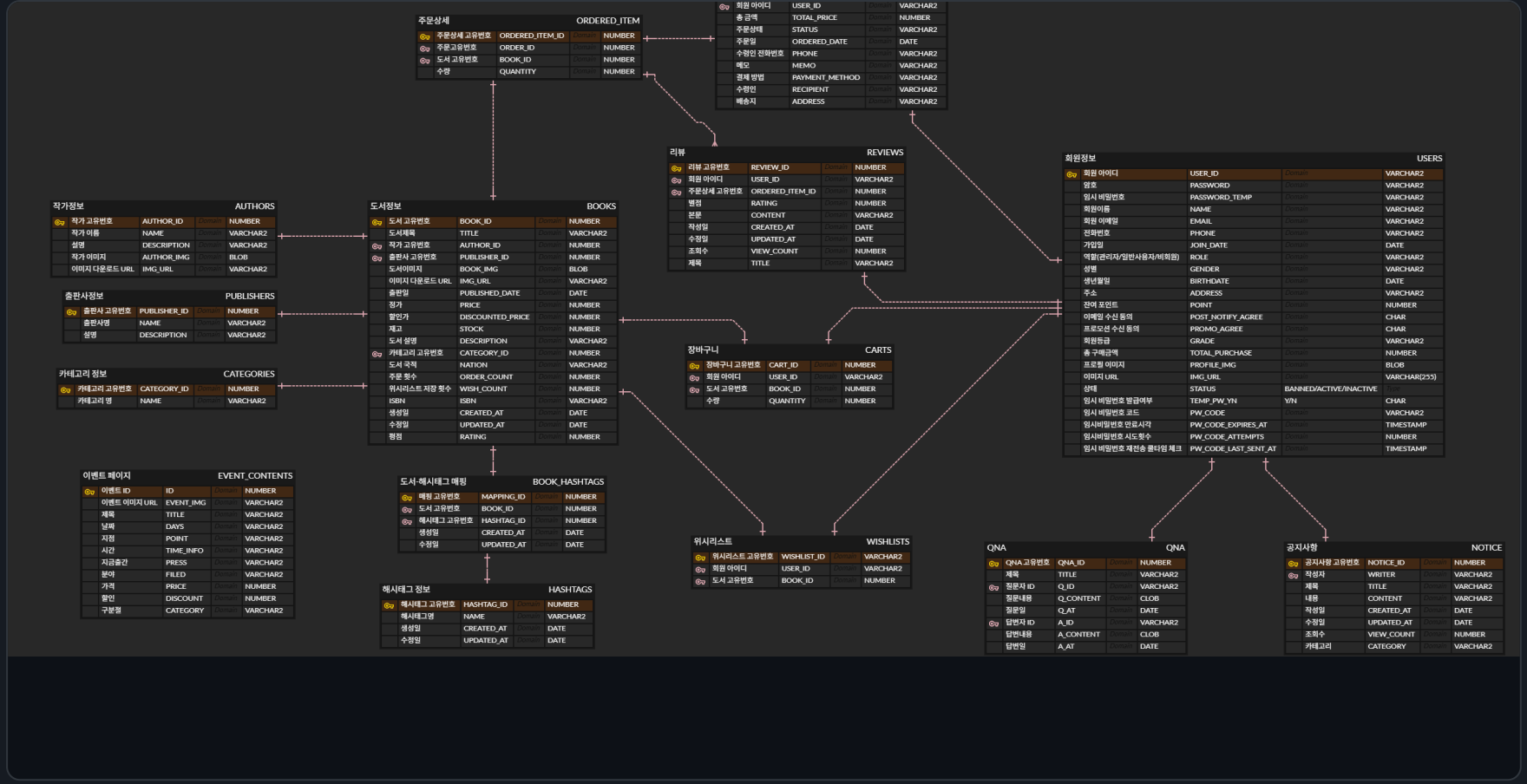
API부터 UI까지 안정적인 도서 커머스 흐름 구축

Spring Boot | JPA | Oracle | PortOne | Python

중간 프로젝트(BLUEMING PAGE)에서 경험한 한계를 개선한 고도화 버전. Mapper 기반 SQL 대신 **JPA/ORM으로 전환**하고, 실제 결제 시스템(PortOne)과 외부 API(알라딘) 연동을 통해 **프로덕션급 백엔드 로직**을 구현했습니다.

- → 반응형 UI (모바일 퍼스트)
- → 소셜 로그인 및 결제 API 통합
- → JPA 기반 ORM 데이터 처리 (Mapper 미사용)
- → 관리자 페이지를 통한 회원·도서·주문 관리
- → PortOne 3단계 결제 위변조 방지 로직
- → 알라딘 API ISBN UPSERT 데이터 파이프라인
- → 마이페이지 SPA (비동기 갱신)
- → Skeleton UI + 페이지네이션

ERD · DATABASE SCHEMA


주문상세 ORDERED_ITEM
주문상세 고유번호 ORDERED_ITEM_ID NUMBER
주문고유번호 ORDER_ID NUMBER
도서 고유번호 BOOK_ID NUMBER
수량 QUANTITY NUMBER
회원 아이디 USER_ID VARCHAR2
총 금액 TOTAL_PRICE NUMBER
주문상태 STATUS VARCHAR2
주문일 ORDERED_DATE DATE
수령인 전화번호 PHONE VARCHAR2
메모 MEMO VARCHAR2
결제 방법 PAYMENT_METHOD VARCHAR2
수령인 RECIPIENT VARCHAR2
배송지 ADDRESS VARCHAR2
리뷰 REVIEWS
리뷰 고유번호 REVIEW_ID NUMBER
회원 아이디 USER_ID VARCHAR2
주문상세 고유번호 ORDERED_ITEM_ID NUMBER
별점 RATING NUMBER
본문 CONTENT VARCHAR2
작성일 CREATED_AT DATE
수정일 UPDATED_AT DATE
조회수 VIEW_COUNT NUMBER
제목 TITLE VARCHAR2
회원정보 USERS
회원 아이디 USER_ID VARCHAR2
암호 PASSWORD VARCHAR2
임시 비밀번호 PASSWORD_TEMP VARCHAR2
회원이름 NAME VARCHAR2
회원 이메일 EMAIL VARCHAR2
전화번호 PHONE VARCHAR2
가입일 JOIN_DATE DATE
역할(관리자/일반사용자/비회원) ROLE VARCHAR2
성별 GENDER VARCHAR2
생년월일 BIRTHDATE DATE
주소 ADDRESS VARCHAR2
잔여 포인트 POINT NUMBER
이메일 수신 동의 POST_NOTIFY_AGREE CHAR
프로모션 수신 동의 PROMO_AGREE CHAR
회원등급 GRADE VARCHAR2
총 구매금액 TOTAL_PURCHASE NUMBER
프로필 이미지 PROFILE_IMG BLOB
이미지 URL IMG_URL VARCHAR(255)
상태 STATUS BANNED/ACTIVE/INACTIVE
임시 비밀번호 발급여부 TEMP_PW_YN Y/N CHAR
임시 비밀번호 코드 PW_CODE VARCHAR2
임시비밀번호 만료시각 PW_CODE_EXPIRES_AT TIMESTAMP
임시비밀번호 시도횟수 PW_CODE_ATTEMPTS NUMBER
임시 비밀번호 재전송 쿨타임 체크 PW_CODE_LAST_SENT_AT TIMESTAMP
작가정보 AUTHORS
작가 고유번호 AUTHOR_ID NUMBER
작가 이름 NAME VARCHAR2
설명 DESCRIPTION VARCHAR2
작가 이미지 AUTHOR_IMG BLOB
이미지 다운로드 URL IMG_URL VARCHAR2
도서정보 BOOKS
도서 고유번호 BOOK_ID NUMBER
도서제목 TITLE VARCHAR2
작가 고유번호 AUTHOR_ID NUMBER
출판사 고유번호 PUBLISHER_ID NUMBER
도서이미지 BOOK_IMG BLOB
이미지 다운로드 URL IMG_URL VARCHAR2
출판일 PUBLISHED_DATE DATE
정가 PRICE NUMBER
할인가 DISCOUNTED_PRICE NUMBER
재고 STOCK NUMBER
도서 설명 DESCRIPTION VARCHAR2
카테고리 고유번호 CATEGORY_ID NUMBER
도서 국적 NATION VARCHAR2
주문 횟수 ORDER_COUNT NUMBER
위시리스트 저장 횟수 WISH_COUNT NUMBER
ISBN ISBN VARCHAR2
생성일 CREATED_AT DATE
수정일 UPDATED_AT DATE
평점 RATING NUMBER
출판사정보 PUBLISHERS
출판사 고유번호 PUBLISHER_ID NUMBER
출판사명 NAME VARCHAR2
설명 DESCRIPTION VARCHAR2
카테고리 정보 CATEGORIES
카테고리 고유번호 CATEGORY_ID NUMBER
카테고리 명 NAME VARCHAR2
장바구니 CARTS
장바구니 고유번호 CART_ID NUMBER
회원 아이디 USER_ID VARCHAR2
도서 고유번호 BOOK_ID NUMBER
수량 QUANTITY NUMBER
이벤트 페이지 EVENT_CONTENTS
이벤트 ID ID NUMBER
이벤트 이미지 URL EVENT_IMG VARCHAR2
제목 TITLE VARCHAR2
날짜 DAYS VARCHAR2
지점 POINT VARCHAR2
시간 TIME_INFO VARCHAR2
지금출간 PRESS VARCHAR2
분야 FILED VARCHAR2
가격 PRICE NUMBER
할인 DISCOUNT NUMBER
구분점 CATEGORY VARCHAR2
도서-해시태그 매핑 BOOK_HASHTAGS
매핑 고유번호 MAPPING_ID NUMBER
도서 고유번호 BOOK_ID NUMBER
해시태그 고유번호 HASHTAG_ID NUMBER
생성일 CREATED_AT DATE
수정일 UPDATED_AT DATE
해시태그 정보 HASHTAGS
해시태그 고유번호 HASHTAG_ID NUMBER
해시태그명 NAME VARCHAR2
생성일 CREATED_AT DATE
수정일 UPDATED_AT DATE
위시리스트 WISHLISTS
위시리스트 고유번호 WISHLIST_ID VARCHAR2
회원 아이디 USER_ID VARCHAR2
도서 고유번호 BOOK_ID NUMBER
QNA QNA
QNA 고유번호 QNA_ID NUMBER
제목 TITLE VARCHAR2
질문자 ID Q_ID VARCHAR2
질문내용 Q_CONTENT CLOB
질문일 Q_AT DATE
답변자 ID A_ID VARCHAR2
답변내용 A_CONTENT CLOB
답변일 A_AT DATE
공지사항 NOTICE
공지사항 고유번호 NOTICE_ID NUMBER
작성자 WRITER VARCHAR2
제목 TITLE VARCHAR2
내용 CONTENT VARCHAR2
작성일 CREATED_AT DATE
수정일 UPDATED_AT DATE
조회수 VIEW_COUNT NUMBER
카테고리 CATEGORY VARCHAR2

04 PortOne 3단계 결제 로직

CORE IMPLEMENTATION · SECURITY CRITICAL

Prepare → Request → Verify: 서버 측 금액 검증을 통한 위변조 방지

> 💡 결제 금액을 클라이언트만 믿으면 안 됩니다. 공격자가 브라우저 개발자 도구에서 결제 금액을 조작하면 실제 결제 금액보다 낮은 금액으로 결제가 완료될 수 있습니다. **서버가 merchantUid와 금액을 사전 등록하고(prepare), 결제 완료 후 서버에서 재검증(verify)하는 3단계 구조**로 이를 원천 차단합니다.

STEP 1 · PREPARE

결제 사전 준비

- 서버가 고유 결제번호 (merchantUid) 생성
- DB에 금액 및 merchantUid 선등록
- 멱등성 기반 확보 · 중복 결제 방지
- 더블 클릭 · 새로고침 오류 차단

```
// async preparePay(total)
```

→

STEP 2 · REQUEST_PAY

PortOne 결제 요청

- PortOne SDK로 실제 결제 창 호출
- 클라이언트는 단순 요청 역할만 담당
- 성공/실패/취소 콜백 처리
- 실패 시 사용자에게 안내 후 종료

```
// IMP.request_pay(payReq)
```

→

🔒 핵심 보안 단계

STEP 3 · VERIFY

서버 검증 · 주문 확정

- PortOne API로 실 결제 금액 재조회
- 서버 사전 등록 금액과 비교 검증
- 금액 불일치 시 결제 취소 처리
- 검증 성공 시 DB에 PAID 주문 생성

```
// POST /payments/verify
```

POST /payments/prepare
→ merchantUid, amount 기록

PortOne SDK 호출
→ rsp.success 여부 판단

금액 위변조 완전 차단
→ PAID 상태 DB 저장

// FULL PAYMENT FLOW DIAGRAM

장바구니 → 결제정보 모달 → prepare API (merchantUid 생성) → PortOne 결제창

결제 성공 시 ↓ 결제 실패 시 → 안내 후 종료

verify API (금액 재검증) → 주문 생성 (PAID 상태 저장) → 마이페이지 이동

⚠️ 위변조 시나리오

클라이언트에서 `amount = 1` 로 조작 후 결제 시도 → 서버 verify 단계에서 PortOne 실 결제금액과 비교 → **불일치 감지 · 자동 취소**

🔄 멱등성 보장

merchantUid에 `UNIQUE` 제약 적용 → 새로고침·지연·더블 클릭으로 인한 중복 결제 원천 차단 → **단 한 번만 결제 승인**

RELATED FEATURE

마이페이지 SPA 구현 (위시리스트 → 장바구니 → 주문내역)

❤️ 위시리스트

- 동일 도서 중복 담기 방지

🛒 장바구니

- 수량 변경 시 즉시 반영 (Ajax)

📦 주문내역

- 주문 상태 변경 로직 구현

- 장바구니 담기 버튼 비동기 처리
- 삭제 버튼 즉시 반영

- 총 금액 / 배송비 자동 계산
- 결제하기 클릭 시 PortOne 연동

- 결제완료/취소 상태 표시
- 결제 취소 시 PortOne 서버 연동

05 알라딘 API 데이터 파이프라인

DATA ENGINEERING · PYTHON + ORACLEDB

Aladin OpenAPI → ISBN UPSERT → Oracle DB 적재 파이프라인

💡 외부 API 데이터를 그냥 INSERT하면 중복 실행 시 데이터가 쌓입니다. **ISBN이라는 도서의 고유 식별자를 기준으로 이미 존재하면 UPDATE, 없으면 INSERT하는 UPSERT 패턴**을 적용해 멱등성 있는 파이프라인을 구축했습니다.

PIPELINE STEPS

1. **Aladin OpenAPI 요청**
   도서 JSON 응답 수신
2. **Python 수집 모듈**
   requests 호출 / JSON 파싱
3. **데이터 정제 (Cleaning)**
   불필요 문자열 제거 · 형식 변환

book_pipeline.py — UPSERT 처리

```python
# ISBN 중복 체크
cursor.execute("SELECT BOOK_ID FROM BOOKS WHERE ISBN = :1",
               (isbn,))
row = cursor.fetchone()

if row:
    # 기존 BOOK 업데이트: IMG_URL만 갱신
    img_url = book.get("cover")
    cursor.execute("""
        UPDATE BOOKS
        SET IMG_URL = :1, UPDATED_AT = SYSDATE
        WHERE ISBN = :2
    """, (img_url, isbn))
else:
    # 새 도서 INSERT
    desc = (book.get("description") or "")[:1000]
```

4 **ISBN 중복 검사 & UPSERT**

UPDATE or INSERT 자동 처리

5 **Oracle DB 저장**

BOOKS / AUTHORS / PUBLISHERS / CATEGORY

```
    cursor.execute("""
        INSERT INTO BOOKS (
            BOOK_ID, TITLE, AUTHOR_ID, PUBLISHER_ID,
            DESCRIPTION, ISBN, PRICE, IMG_URL,
            CREATED_AT, UPDATED_AT
        ) VALUES (
            SEQ_BOOKS.NEXTVAL, :1, :2, :3,
            :4, :5, :6, :7, SYSDATE, SYSDATE
        )
    """, (title, author_id, pub_id,
              desc, isbn, price, img_url))
```

book_pipeline.py — 데이터 정제 로직

```python
# 저자명 정제: 불필요한 괄호·직함 제거
def clean_author_name(name: str) -> str:
    if not name or name.strip() == "":
        return "미상"
    # 불필요한 괄호 내용 제거 (영문 병기 형태)
    name = re.sub(r'\s*\([^)]*[가-힣][^)]*\)',
                  "", name)
    return name.strip()

# 카테고리명 정제: ▶ 구분자 이후 추출
def clean_category(cat_name: str) -> str:
    if '>' in cat_name:
        return cat_name.split('>')[-1].strip()
    return cat_name.strip()
```

⚡ **UPSERT 로직 흐름**

→ → → row 존재 → UPDATE
IMG_URL 갱신 →

Aladin API 응답 → ISBN 추출 → SELECT BOOKS WHERE ISBN = ? → row 없음 → INSERT 신규 도서 등록 → Oracle DB 반영

🔥 ORA-12899 오류 해결 경험

알라딘 API 응답 중 일부 도서 설명(description)과 카테고리명이 `VARCHAR2` 한도 초과 → **ORA-12899: value too large** 오류 발생 → DBA 협의 후 해당 컬럼을 `CLOB` 타입으로 변경 → **DB 구조 개선으로 안정적인 데이터 적재 환경 구축**

06 시스템 안정성을 위한 문제 해결

💳 PortOne 결제 멱등성 문제

PROBLEM

> 새로고침 또는 네트워크 지연 시 동일한 결제 요청이 중복으로 처리되는 문제 발생

CAUSE

> merchantUid 중복 검증 로직 부재로 인해 동일 결제 요청이 여러 번 처리될 수 있었음

SOLUTION

> merchantUid를 기준으로 **UNIQUE 제약 키** 적용으로 중복 결제 원천 차단 → 단 한 번만 결제가 승인되도록 로직 개선

📚 Aladin API 데이터 적재 오류

PROBLEM

> API 응답 데이터 삽입 시 `ORA-12899: value too large` 오류 발생

CAUSE

> 카테고리명과 도서 설명(description) 데이터가 VARCHAR2 컬럼 한도를 초과하여 삽입 불가

SOLUTION

> DBA 협의 후 해당 컬럼을 **CLOB 타입**으로 변경 → DB 구조 개선으로 안정적인 데이터 적재 환경 구축

07 지속적인 성장 및 인프라 관리

// 환경에 의존하지 않고 스스로 환경을 만드는 개발자

"학원에서 배운 것에 그치지 않고, 이론 기반을 쌓고 스스로 개발 환경을 구축하며 인프라 역량을 확장하고 있습니다."

로컬 서버 환경 재구축

KH 정보교육원 수료 후, 학원 인프라에 의존하지 않고 **개인 PC 환경에 Oracle DB와 Spring Boot 개발 환경을 독립적으로 재구성**했습니다. 단순히 배운 것을 따라하는 것이 아닌, 환경 문제를 스스로 진단하고 해결하는 과정을 통해 실제 서버 운영에 대한 감각을 체득했습니다.

Oracle DB 로컬 설치 | Spring Boot 독립 구성 | 환경 적응력

정보처리기사 필기 합격 (2026)

실무 중심의 개발 학습과 병행하여 **CS 이론적 기반을 체계적으로 정립**했습니다. 자료구조, 운영체제, 네트워크, 데이터베이스, 소프트웨어 공학 등 핵심 CS 과목을 학습함으로써 코드 이면의 원리를 이해하는 개발자로 성장하고 있습니다.

자료구조 | 운영체제 | 네트워크 | DB 설계 | SW 공학

향후 인프라 확장 계획

현재 로컬 중심의 개발 환경을 넘어 **Docker 컨테이너화 및 AWS EC2 클라우드 배포**를 통해 실제 운영 환경에서의 서비스 배포 역량을 갖출 계획입니다. 인프라를 이해하는 백엔드 개발자로 성장하는 것을 목표로 합니다.

Docker | AWS EC2 | CI/CD | 클라우드 배포

// GROWTH ROADMAP

KH 교육 수료
2025.10

로컬 환경 재구축
Oracle + Spring Boot

정보처리기사 필기
2026 합격

정보처리기사 실기
준비 중

Docker 컨테이너화
진행 예정

AWS EC2 배포
진행 예정

신승화

$ Backend Developer

010.7726.7867 | tlstmdghk2@naver.com | GitHub

문제를 해결하는 개발자 · 인프라를 이해하는 백엔드 엔지니어 · 계속 성장하는 사람